

Project Description

Inspired by Stardew Valley's fishing minigame, we created a game where the user controls the reel of a fishing rod (using the PS2 keyboard as input) to keep the fish within a bar (see brown bar on the right of Figure 1) until a progress bar (see yellow bar in Figure 1) fills up. The challenge is that the fish moves up and down sporadically and when it isn't in the brown bar, the progress bar goes down. Hitting 0 on the progress bar ends the game, taking the user back to the main screen, generating a new fish and resetting their score while logging the current highest score. To add layers of difficulty, each "fish" was random and demonstrated through increasingly sporadic movements. Specifically, the direction and speed of the fish were randomized, in periods that were also randomized, which makes it nearly impossible for the user to predict the fish behaviour, making it a game of reflexes. Our game is displayed on VGA and we implemented sound effects (by generating waves of predetermined frequencies) to indicate when a fish was caught, or when the game ended.



Figure 1. Game screenshot

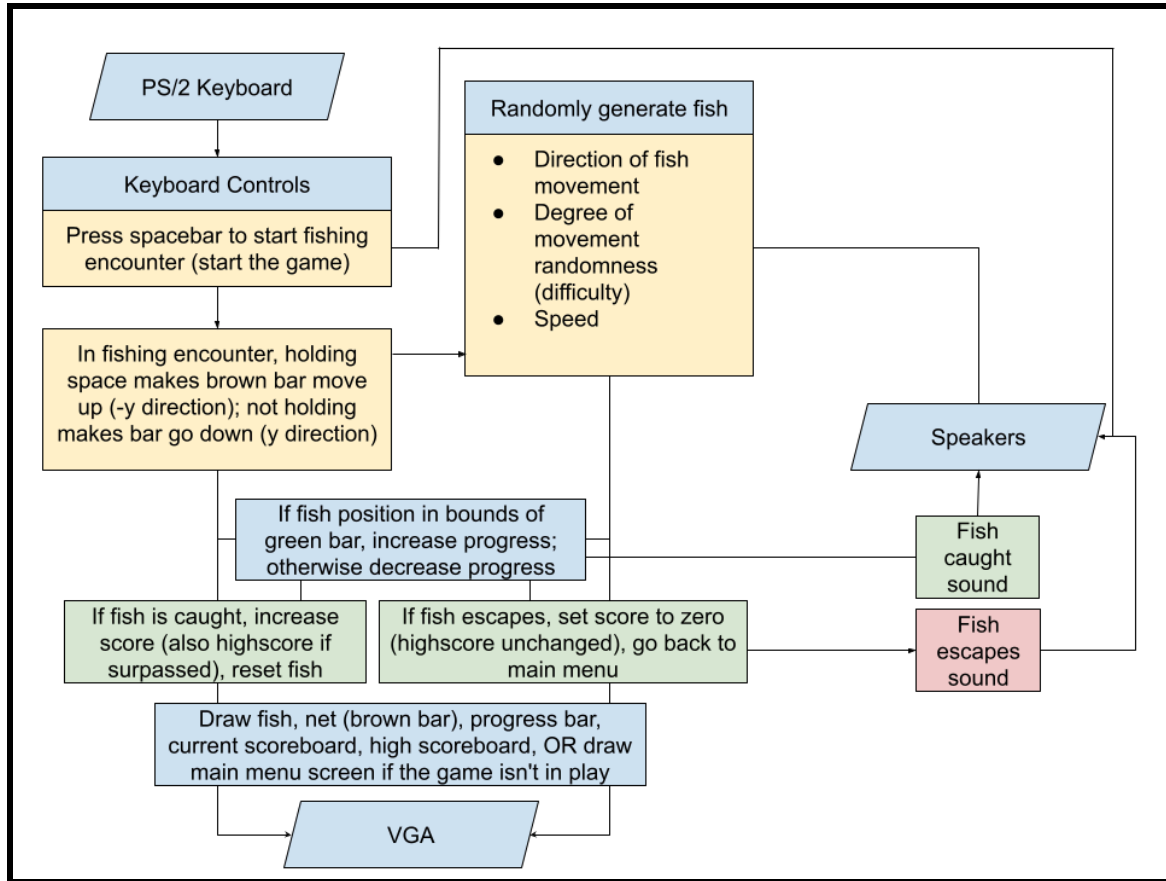


Figure 2. Updated block diagram

Attribution Table

Rashaad Anwar	50%	Gilberto	50%
Week 1			
Spawns fish with randomized difficulties by randomly generating a speed and movement frequency		Animating fish to VGA (using rectangles as placeholder graphics)	
Week 2			
Keyboard input for user catching fish		Keyboard input for user catching fish	
Made VGA code more robust by creating functions to draw and erase specific elements/sprites rather than clearing the whole screen and redrawing everything		Progress bar for indicating fish catch completion (VGA)	
Added new images for graphics (start screen, game background, fish)		Score system (not displayed numerically yet)	
		Auditory indication of a catch (speaker output)	

	Loop game when a fish is caught (using game states)
Week 3	
Increased code efficiency to allow for faster polling (by updating VGA purely when necessary) Added high score tracking and display Cleaned up UI design Adjusted game difficulty (fish spawn code) Combined previous score mechanic, audio function, and game state system to add the ability to lose the game, setting the current score to 0, updating the high score if necessary, then resetting it to the start state waiting for the user to restart the game	Increased code efficiency to allow for faster polling (by updating VGA purely when necessary) Displaying current score from 0 to 99 numerically Added more updated graphics, replacing the simple placeholder shapes like boxes with actual graphics